高并发内存池项目说明

一、项目整体流程介绍

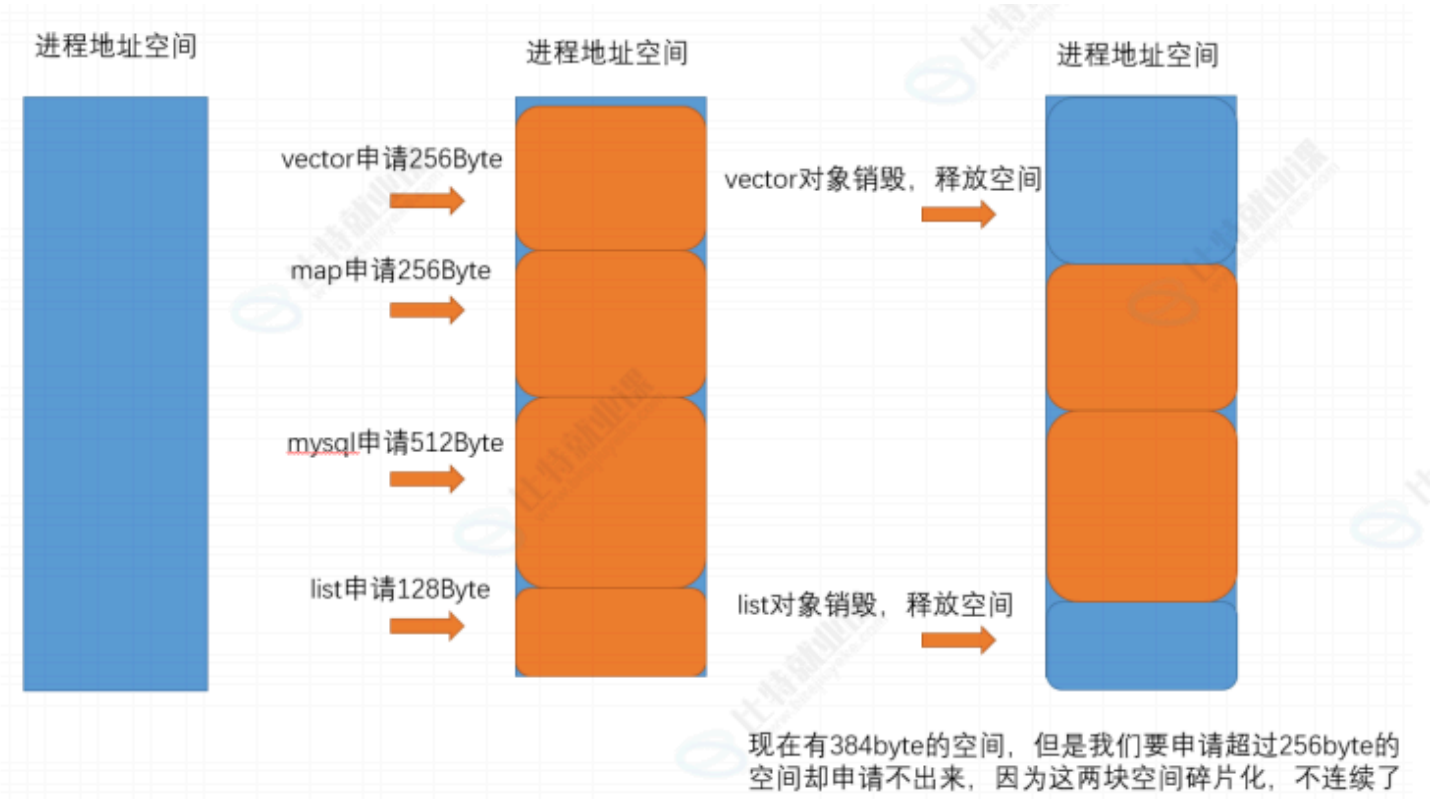
这个项目是一个基于 C++ 实现的高并发内存分配器，整体设计借鉴了 TCMalloc 的分层缓存思想。项目主要面向多线程环境下频繁申请和释放小块内存的场景。

传统的 `malloc` 和 `free` 本身是线程安全的，但是在多线程高并发场景下，如果大量线程频繁申请和释放小块内存，仍然可能产生三个问题：

第一是多个线程频繁访问共享内存分配器，容易产生锁竞争；

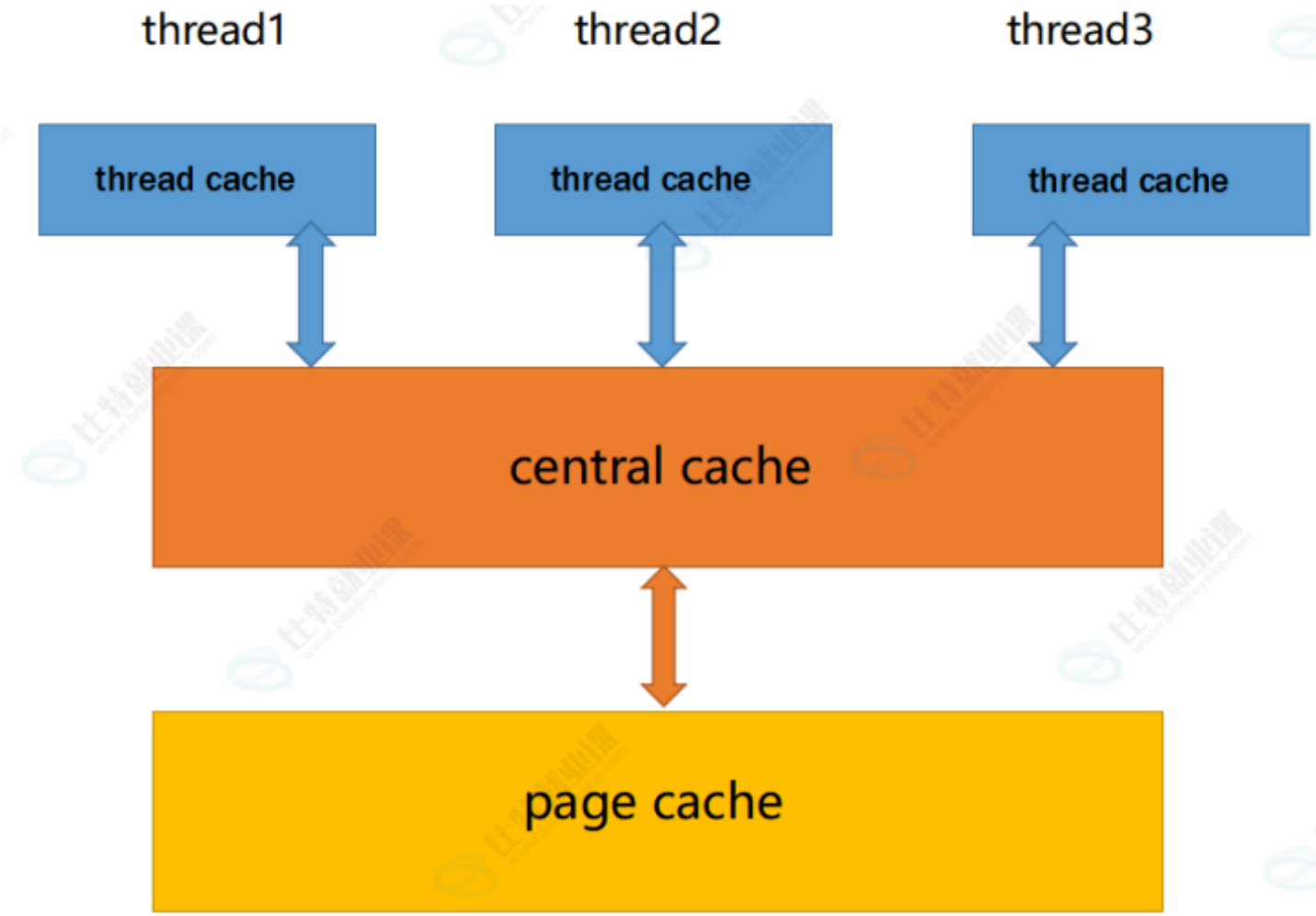
第二是频繁向操作系统申请和释放内存，系统调用开销比较大；

第三是不同大小的内存块不断申请和释放，容易产生内存碎片，这里的内存碎片包括内碎片和外碎片。



针对这些问题，我设计了 ThreadCache、CentralCache 和 PageCache 三级缓存架构。

其中，ThreadCache 是线程私有缓存，主要负责高频小对象的快速申请和释放；CentralCache 是所有线程共享的中心缓存，负责在线程之间调度小对象；PageCache 是页级缓存，负责连续页的申请、切分、回收和合并。PageCache 再向底层操作系统申请实际内存。



二、内存大小的分类与映射关系

在介绍三级缓存之前，需要先说明一下项目如何管理不同大小的内存。

项目把不超过 256KB 的内存定义为小对象。对于小对象，不是为每一种字节大小都建立一个自由链表，而是按照一定的对齐规则划分为 208 个大小类别。

具体划分为五个区间：

代码块

```
1B～128B：按照8字节对齐，共16个桶

129B～1KB：按照16字节对齐，共56个桶

1KB+1B～8KB：按照128字节对齐，共56个桶

8KB+1B～64KB：按照1KB对齐，共56个桶

64KB+1B～256KB：按照8KB对齐，共24个桶
```

总桶数为：16 + 56 + 56 + 56 + 24 = 208 个桶

这样做的目的是在自由链表数量和内碎片之间取得平衡。

对齐粒度会随着对象大小增加而增加，整体可以把对齐产生的内碎片控制在大约10%左右。

举个例子：

代码块

```
哈希桶对应位置如下：
    8
    16
    24
    ...
    128
    144
    160
    ...
    1024
    1152
    1280
    ...
    8*1024
    9*1024
    ...
    64*1024
    72*1024
    ...
    256*1024

```

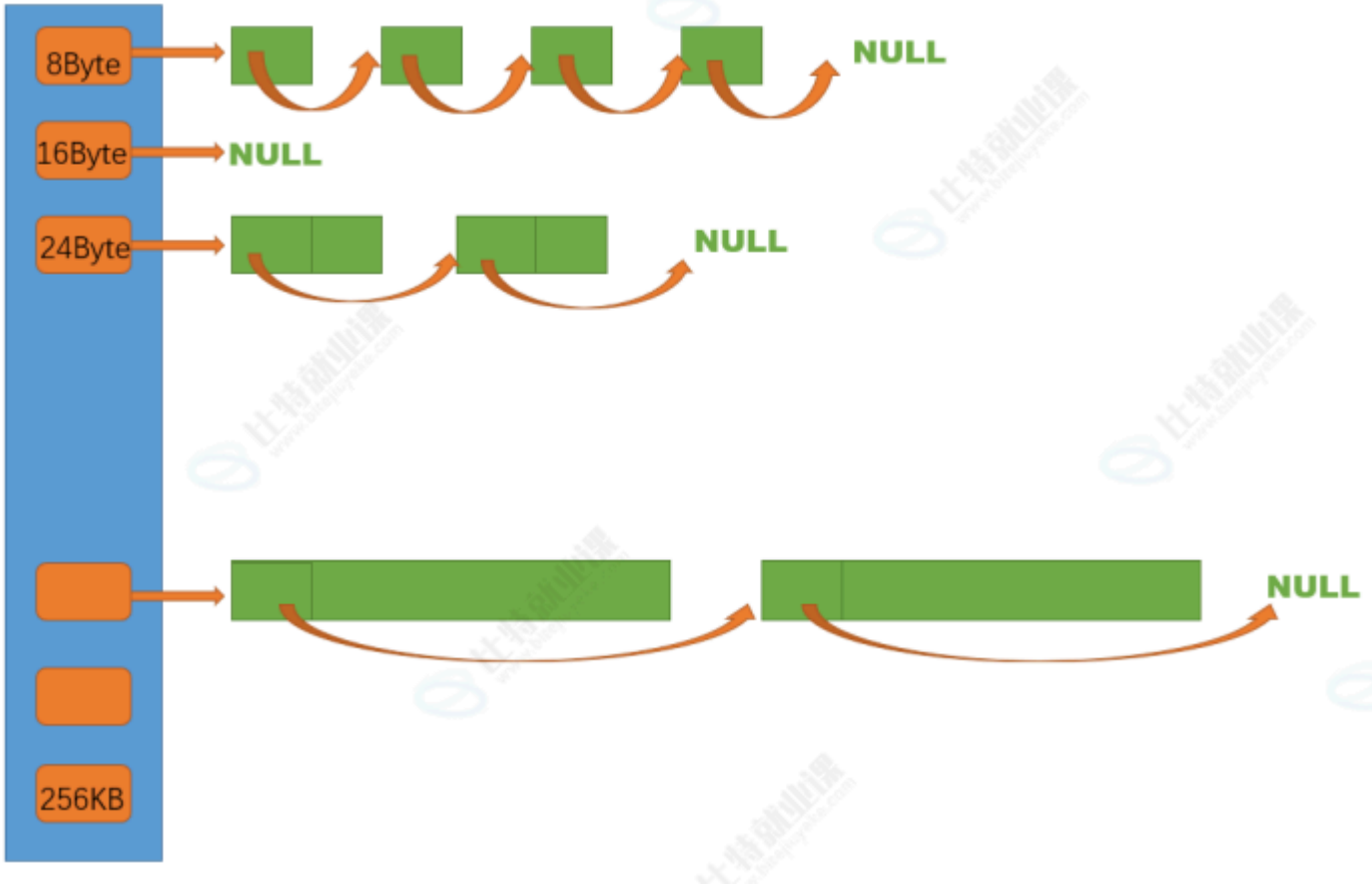
以申请129字节的对象，会被对齐到144字节，映射到对应的自由链表桶
(144-129)/144约等于0.104，说明有约10%的内碎片浪费

三、ThreadCache

ThreadCache 是三级缓存的第一层，也是距离用户最近的一层。

每个线程都通过 TLS，也就是线程局部存储，拥有独立的 ThreadCache。ThreadCache 内部包含208个 FreeList，每个 FreeList 对应一种对象大小。

因为不同线程操作的是各自独立的 ThreadCache，所以在线程本地申请和释放内存时不需要加锁。这是整个项目提高并发性能的关键的一个地方。



3.1ThreadCache申请内存

内存申请的统一入口是：

代码块

```
static void* ConcurrentAlloc(size_t size)
{
    //大于256kb直接向堆申请
    if(size>MAX_BYTES)
    {
        size_t alignSize=SizeClass::RoundUp(size);
        size_t kpage=alignSize>>PAGE_SHIFT;

        PageCache::GetInstance()->_pageMtx.lock();
        Span* span=PageCache::GetInstance()->NewSpan(kpage);
        PageCache::GetInstance()->_pageMtx.unlock();

        void* ptr=(void*)(span->_pageId<<PAGE_SHIFT);

        return ptr;
    }
    // 通过TLS 每个线程无锁的获取自己的专属的ThreadCache对象
    if (pTLSThreadCache == nullptr)
    {
        static ObjectPool<ThreadCache> _threadCachePool;
```

```
21          pTLSThreadCache = _threadCachePool.New();
22          //pTLSThreadCache = new ThreadCache;
23      }
24
25      //cout << std::this_thread::get_id() << ":"<<pTLSThreadCache<<endl;
26
27      return pTLSThreadCache->Allocate(size);
28  }
```

如果当前线程第一次申请内存，就创建一个属于该线程的 ThreadCache（可以用定长内存池优化这里的申请过程，优化后是创建一个_threadCachePool）。然后进入：Allocate(size)

代码块

```
1   void* ThreadCache::Allocate(size_t size)
2   {
3       assert(size <= MAX_BYTES);
4       size_t alignSize = SizeClass::RoundUp(size);// 对齐后的大小，比如size=9，会给对齐到16
5       size_t index = SizeClass::Index(size);//对应自由链表所在桶的索引，比如size=9，会计算出是第2个桶
6
7       if (!_freeLists[index].Empty())
8       {
9           return _freeLists[index].Pop();
10      }
11      else
12      {
13          return FetchFromCentralCache(index, alignSize);
14      }
15  }
```

在这个函数中，首先通过 `RoundUp` 得到对齐后的大小，

代码块

```
1   static inline size_t RoundUp(size_t size)
2       {
3           // 根据对象大小选择合适的对齐方式
4           if (size <= 128)
5           {//这里首先用8字节对齐，因为64位系统下指针大小为8字节，保证每个空闲内存块至少能存储一个指针的大小
6               return _RoundUp(size, 8);
7           }
8           else if (size <= 1024)
9           {
```

```
            return _RoundUp(size, 16);
        }
        else if (size <= 8*1024)
        {
            return _RoundUp(size, 128);
        }
        else if (size <= 64*1024)
        {
            return _RoundUp(size, 1024);
        }
        else if (size <= 256 * 1024)
        {
            return _RoundUp(size, 8*1024);
        }
        else// size > 256 * 1024，以页为单位对齐,这里写成1<<PAGE_SHIFT是为了方便以后修改页大小，假设页大小改为16KB，只需要修改PAGE_SHIFT的值即可
        {
            return _RoundUp(size, 1<<PAGE_SHIFT);
        }
    }


    static inline size_t _RoundUp(size_t bytes, size_t alignNum)
    {
        // 将bytes上调到alignNum的整数倍
        // return ((bytes + alignNum - 1) & ~(alignNum - 1));
        //下面写法和上面注释掉的位运算写法效果一样，只是更直观

        // 已经是 alignNum 的整数倍，直接返回
        if (bytes % alignNum == 0)
        {
            return bytes;
        }
        // 否则补到下一个 alignNum 的整数倍
        return (bytes / alignNum + 1) * alignNum;
        //举个例子_RoundUp(129, 16) = 144，说明申请129字节的对象会被对齐到144字节
        //129%16! =0
        // (129/16+1) *16= (8+1) *16=144

        //_RoundUp(9, 8); // 结果为16，说明申请9字节的对象会被对齐到16字节
        //9%8! =0，说明9不是8的整数倍
        //(9/8+1)*8 = 16，说明9字节被对齐到16字节
    }
```

再通过 `Index` 找到对应的自由链表桶。

代码块

```
// 计算映射的哪一个自由链表桶
    static inline size_t Index(size_t bytes)
    {
        assert(bytes <= MAX_BYTES);

        static int group_array[5] = { 16, 56, 56, 56, 24 };
        // group_array记录每个区间的链表数量
        // [1,128] 8字节对齐 16个桶
        // [129,1024] 16字节对齐 56个桶
        // [1025,8*1024] 128字节对齐 56个桶
        // [8*1024+1,64*1024] 1024字节对齐 56个桶
        // [64*1024+1,256*1024] 8*1024字节对齐 24个桶
        if (bytes <= 128){
            // 8字节对齐，索引从0开始
            //举个例子
            //_Index(128, 3)
            // = ((128 + 8 - 1) >> 3) - 1
            // = (135 >> 3) - 1
            // = 16 - 1
            // = 15，对应自由链表的第15个桶
            return _Index(bytes, 3);
        }
        else if (bytes <= 1024){
            // 16字节对齐，索引从16开始
            // 举个例子
            //_Index(129-128, 4)
            // = ((1 + 16 - 1) >> 4) - 1
            // = (16 >> 4) - 1
            // = 1 - 1
            // = 0
            //0+group_array[0] = 16，对应自由链表的第16个桶
            return _Index(bytes - 128, 4) + group_array[0];
        }
        else if (bytes <= 8 * 1024){
            // 128字节对齐，索引从72开始
            return _Index(bytes - 1024, 7) + group_array[1] + group_array[0];
        }
        else if (bytes <= 64 * 1024){
            // 1024字节对齐，索引从128开始
            return _Index(bytes - 8 * 1024, 10) + group_array[2] +
group_array[1] + group_array[0];
        }
        else if (bytes <= 256 * 1024){
            // 8*1024字节对齐，索引从184开始
            return _Index(bytes - 64 * 1024, 13) + group_array[3] +
group_array[2] + group_array[1] + group_array[0];
        }
```

```
46          else{
47              assert(false);
48          }
49
50          return -1;
51      }
```

找到对应的哈希桶之后，如果哈希桶上的自由链表FreeList 不为空，就直接从对应哈希桶的自由链表头部取出一个对象并返回。这个过程只涉及当前线程自己的自由链表，不访问共享数据，也不需要加锁，所以这是项目中速度最快的内存申请路径。

代码块

```
1   void* ThreadCache::Allocate(size_t size)
2   {
3       assert(size <= MAX_BYTES);
4       size_t alignSize = SizeClass::RoundUp(size);// 对齐后的大小
5       size_t index = SizeClass::Index(size);// 对应自由链表所在桶的索引
6
7       if (!_freeLists[index].Empty())
8       {
9           return _freeLists[index].Pop();
10      }
11      else
12      {
13          return FetchFromCentralCache(index, alignSize);
14      }
15  }
```

如果对应的 FreeList 为空，说明 ThreadCache 中没有这个大小的空闲对象，此时就调用：FetchFromCentralCache();向 CentralCache 批量申请对象。

代码块

```
1   void* ThreadCache::FetchFromCentralCache(size_t index, size_t size)
2   {
3       // 慢开始反馈调节算法
4       // 1、最开始不会一次向central cache一次批量要太多，因为要太多了可能用不完
5       // 2、如果你不要这个size大小内存需求，那么batchNum就会不断增长，直到上限
6       // 3、size越大，一次向central cache要的batchNum就越小
7       // 4、size越小，一次向central cache要的batchNum就越大
8       size_t batchNum = std::min(_freeLists[index].MaxSize(),
    SizeClass::NumMoveSize(size));
9       if (_freeLists[index].MaxSize() == batchNum)
10      {
```

```
            _freeLists[index].MaxSize() += 1;
        }

        void* start = nullptr;
        void* end = nullptr;
        size_t actualNum = CentralCache::GetInstance()->FetchRangeObj(start, end,
    batchNum, size);
        assert(actualNum > 0);

        if (actualNum == 1)
        {
            assert(start == end);
            return start;
        }
        else
        {
            _freeLists[index].PushRange(NextObj(start), end, actualNum-1);
            return start;
        }
    }
```

这里我们采用的是慢开始反馈调节算法

ThreadCache 不会每次只向 CentralCache 申请一个对象。因为如果只申请一个，那么下一次再申请相同大小的对象时，还需要重新访问 CentralCache 并加锁。

所以项目采用批量获取策略，一次从 CentralCache 获取多个对象。CentralCache 返回一批对象以后，第一个对象直接交给用户，剩余对象挂到当前线程的 FreeList 中。

这样后续再申请相同大小的对象时，就可以直接从本地 FreeList 获取。

每个 FreeList 中维护一个 `MaxSize`，初始值比较小。ThreadCache 向 CentralCache 申请对象时，实际批量数为：min(MaxSize, NumMoveSize(size));

代码块

```
    // 一次thread cache从中心缓存获取多少个
        static size_t NumMoveSize(size_t size)
        {
            assert(size > 0);

            // [2, 512]，一次批量移动多少个对象的(慢启动)上限值
            // 小对象一次批量上限高
            // 小对象一次批量上限低
            int num = MAX_BYTES / size;
            if (num < 2)
```

```
            num = 2;

        if (num > 512)
            num = 512;

        return num;
    }
```

如果当前线程持续申请同一种大小的对象，`MaxSize` 就会逐渐增长，直到达到这个大小类别允许的批量上限。

这样设计主要是为了平衡两个问题。

如果一开始批量申请太多，线程可能用不完，导致大量空闲内存长期停留在某一个线程中；

如果每次申请太少，又会频繁访问 CentralCache，增加锁竞争。

所以慢启动策略是在“减少锁竞争”和“避免线程囤积内存”之间做平衡。

同时，对象越小，一次批量移动的数量越多；对象越大，一次批量移动的数量越少。

3.2ThreadCache释放内存

内存释放的统一入口ConcurrentFree(ptr, size);

代码块

```
static void ConcurrentFree(void* ptr, size_t size)
{
    if(size>MAX_BYTES)
    {
        //根据内存对象地址，算出对应的页号，
        Span* span=PageCache::GetInstance()->MapObjectToSpan(ptr);
        PageCache::GetInstance()->_pageMtx.lock();
        //然后查找页号对应的span
        PageCache::GetInstance()->ReleaseSpanToPageCache(span);
        PageCache::GetInstance()->_pageMtx.unlock();

        return;
    }
    else
    {
        assert(pTLSThreadCache);
        pTLSThreadCache->Deallocate(ptr, size);
```

```
    }
}
```

超过256KB的小对象，说明不是走的内存池申请的，会直接走ReleaseSpanToPageCache（span）

对于不超过256KB的小对象，会调用：ThreadCache::Deallocate(ptr, size);

代码块

```
void ThreadCache::Deallocate(void* ptr, size_t size)
{
    assert(ptr);
    assert(size <= MAX_BYTES);

    // 找对映射的自由链表桶，对象插入进入
    size_t index = SizeClass::Index(size);
    _freeLists[index].Push(ptr);

    // 当链表长度大于一次批量申请的内存时就开始还一段list给central cache
    if (_freeLists[index].Size() >= _freeLists[index].MaxSize())
    {
        ListTooLong(_freeLists[index], size);
    }
}
```

首先根据传入的 size 计算对应的自由链表桶，然后通过头插法，把对象放回当前线程的 FreeList。

因为 FreeList 属于当前线程，所以这个过程同样不需要加锁。

对象释放回来后，不会立即归还 CentralCache，而是优先保留在线程本地。这样当前线程下一次申请相同大小的对象时，可以直接复用。

但是，如果某一个 FreeList 中的对象数量越来越多，就说明当前线程可能暂时不需要这么多对象。如果一直留在 ThreadCache，会造成单个线程囤积内存。

因此，当 FreeList 的长度达到 `MaxSize` 时，会调用：ListTooLong();

一次从本地链表中取出一批对象，然后批量归还给 CentralCache。

代码块

```
void ThreadCache::ListTooLong(FreeList& list, size_t size)
{
    void* start = nullptr;
    void* end = nullptr;
```

```
    list.PopRange(start, end, list.MaxSize());
    //从start位置取到最后，每个对象size大小。这条链表还给centralcache
    CentralCache::GetInstance()->ReleaseListToSpans(start, size);
}

//取n个对象（头删）
void PopRange(void*& start, void*& end, size_t n)
{
    assert(n <= _size);
    start = _freeList;
    end = start;

    for (size_t i = 0; i < n - 1; ++i)
    {
        end = NextObj(end);
    }

    _freeList = NextObj(end);
    NextObj(end) = nullptr;
    _size -= n;
}
```

3.3ThreadCache核心逻辑

所以 ThreadCache 的核心逻辑可以总结为：

代码块

```
申请时：
本地有对象就直接取
本地没有对象就批量补

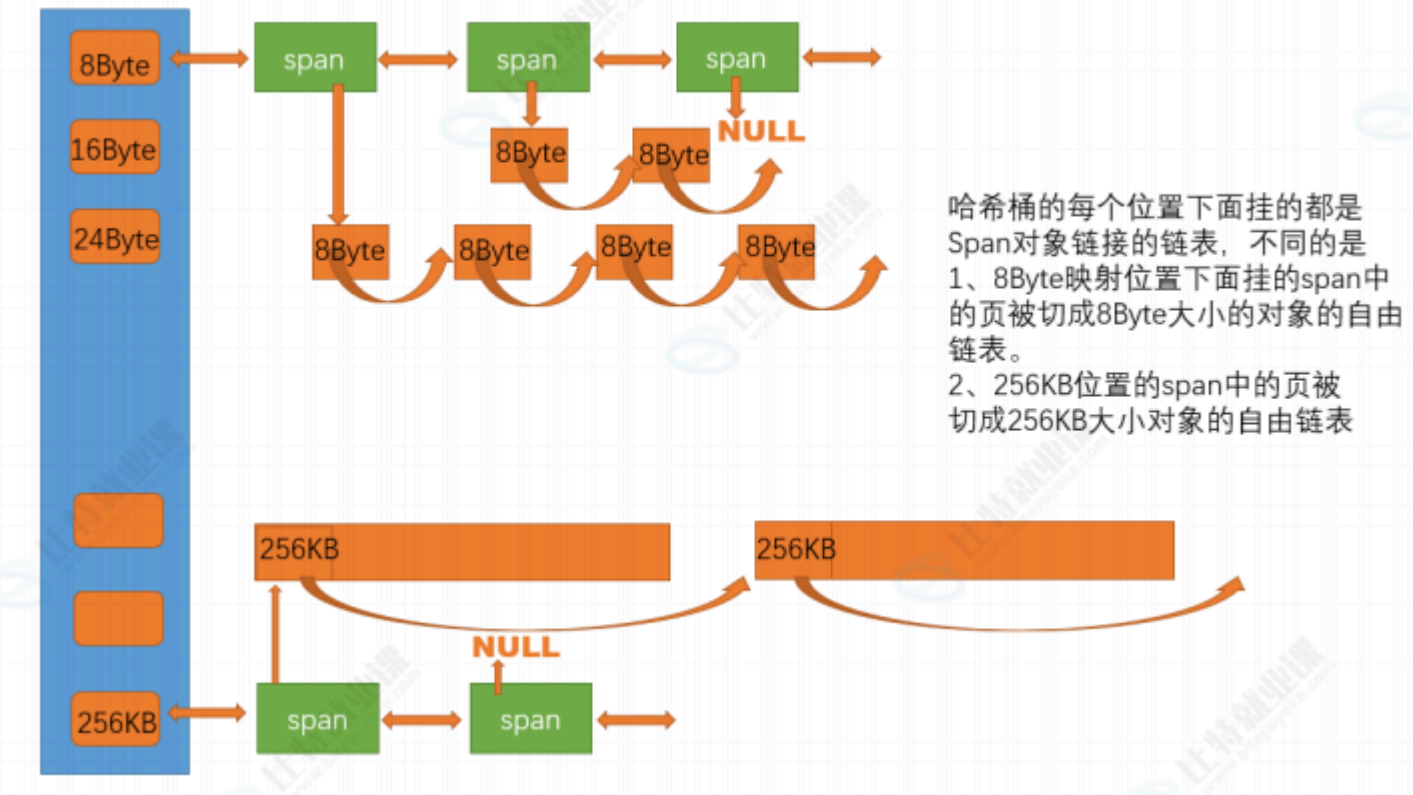
释放时：
优先留在线程本地
本地对象过多就批量归还
```

四、CentralCache

CentralCache 是三级缓存的第二层，它是所有线程共享的。

CentralCache 内部同样按照208个大小类别维护208个 SpanList。

每一个 SpanList 中保存若干个 Span，这些 Span 都被切分成相同大小的小对象。



代码块

```
// 管理多个连续页大块内存跨度结构
struct Span
{
    PAGE_ID _pageId = 0; // 大块内存起始页的页号
    size_t  _n = 0;      // 页的数量

    Span* _next = nullptr;  // 双向链表的结构
    Span* _prev = nullptr;

    size_t _useCount = 0; // 切好小块内存，被分配给thread cache的计数
    void* _freeList = nullptr;  // 切好的小块内存的自由链表

    bool _isUse = false;           // 当前span是否在被使用（用于pagecache合并内存时判断）
};

// 带头双向循环链表
class SpanList
{
public:
    SpanList()
    {
        _head = new Span;
        _head->_next = _head;
        _head->_prev = _head;
```

```
25      }
26
27      Span* Begin()
28      {
29          return _head->_next;
30      }
31
32      Span* End()
33      {
34          return _head;
35      }
36
37      bool Empty()
38      {
39          return _head->_next == _head;
40      }
41
42      void PushFront(Span* span)
43      {
44          Insert(Begin(), span);
45      }
46
47      Span* PopFront()
48      {
49          Span* front = _head->_next;
50          Erase(front);
51          return front;
52      }
53
54      void Insert(Span* pos, Span* newSpan)
55      {
56          assert(pos);
57          assert(newSpan);
58
59          Span* prev = pos->_prev;
60          // prev newspan pos
61          prev->_next = newSpan;
62          newSpan->_prev = prev;
63          newSpan->_next = pos;
64          pos->_prev = newSpan;
65      }
66
67      void Erase(Span* pos)
68      {
69          assert(pos);
70          assert(pos != _head);
71
```

```
72          Span* prev = pos->_prev;
73          Span* next = pos->_next;
74
75          prev->_next = next;
76          next->_prev = prev;
77      }
78
79  private:
80      Span* _head;
81  public:
82      std::mutex _mtx; // 桶锁
83  };
84
```

CentralCache 的作用主要有两个：

第一，为 ThreadCache 批量提供小对象；

第二，接收 ThreadCache 批量归还的小对象。

4.1CentralCache的桶锁设计

因为 CentralCache 是所有线程共享的，所以需要考虑线程安全。

项目没有给整个 CentralCache 使用一把全局锁，而是为每一个哈希桶上的 SpanList 单独设置一把桶锁。

例如，一个线程正在申请32字节对象，另一个线程正在申请128字节对象，它们访问的是不同的 SpanList，申请不会有冲突，因此可以并行执行。

只有两个线程同时操作相同大小类别的对象时，才会竞争同一把桶锁。

这种设计比直接使用一把全局锁的锁粒度更小，可以减少不同大小对象之间不必要的锁竞争。

4.2CentralCache申请内存

ThreadCache 本地自由链表为空时，会调用：CentralCache::FetchRangeObj();

代码块

```
1  // 从中心缓存获取batchNum数量的size大小的对象给thread cache
2  size_t CentralCache::FetchRangeObj(void*& start, void*& end, size_t batchNum,
   size_t size)
3  {
4      size_t index = SizeClass::Index(size);
5      _spanLists[index]._mtx.lock();
6
```

```
7       Span* span = GetOneSpan(_spanLists[index], size);
8       assert(span);
9       assert(span->_freeList);
10
11      // 从span中获取batchNum个对象
12      // 如果不够batchNum个，有多少拿多少
13      start = span->_freeList;
14      end = start;
15      size_t i = 0;
16      size_t actualNum = 1;
17      while ( i < batchNum - 1 && NextObj(end) != nullptr)
18      {
19          end = NextObj(end);//读取当前 end 对象保存的下一个对象地址,将这个地址赋给 end
20          ++i;
21          ++actualNum;
22      }
23      //中间一段自由链表对象取走
24      //链表头连上后面一段
25      span->_freeList = NextObj(end);
26      //被取走的链表断开
27      NextObj(end) = nullptr;
28      //span中被分割的小对象计数增加
29      span->_useCount += actualNum;
30
31      _spanLists[index]._mtx.unlock();
32
33      return actualNum;
34  }
35
36  static void*& NextObj(void* obj)// 获取obj的下一个对象指针，返回对下一个对象指针的引
    用
37  {//obj下一个对象指针在obj所占内存的起始位置，用来存储自由链表中下一个对象的地址
38      //32位系统下是4字节，64位系统下是8字节，保证每个空闲内存块至少能存储一个指针的大小
39      //ps：void*不能直接做 ++ 或 +1 因为 void 表示“未知类型”，编译器不知道它指向的对象
    大小，也就无法决定该移动多少字节。
40      //void* 主要适合“先保存一个通用地址”；需要访问、移动或做地址运算时，再转回具体指针类
    型。
41      return *(void**)obj;
42  }
```

CentralCache 首先根据对象大小找到对应的 SpanList，然后获取这个桶的互斥锁。

遍历当前 SpanList，寻找一个 `_freeList` 不为空的 Span。

Span 可以理解为一段连续页组成的大块内存。CentralCache 会把这段大块内存按照固定对象大小切分成多个小对象，并使用自由链表连接起来。

如果找到还有空闲对象的 Span，就从这个 Span 的自由链表中摘出最多 `batchNum` 个对象，然后返回给 ThreadCache。

实际取出多少对象，就把 Span 的 `_useCount` 增加多少。

调用GetOneSpan()流程：

代码块

```
// 从 CentralCache 的某个桶中，找到一个还有空闲小对象的 span；
//如果没有，就向 PageCache 申请一个 span，把它切成多个小对象，再挂回当前桶
Span* CentralCache::GetOneSpan(SpanList& list, size_t size)
{
    // 查看当前的spanlist中是否有还有未分配对象的span
    //list 是当前大小类别对应的 SpanList。一个 span 管理一段连续页，并被切成多个固定大小的小对象。
    Span* it = list.Begin();
    while (it != list.End())
    {
        //每个span里面有_freeList指针，指向切好的小块内存的自由链表的第一个对象，
        //如果不为空，说明该span还有未分配对象
        if (it->_freeList != nullptr)
        {
            return it;
        }
        else
        {
            it = it->_next;//找当前spanlist的下一个span
        }
    }
    // 走到这里说明当前的spanlist中没有还有未分配对象的span，需要去page cache中获取一个新的span

    // 遍历完对应size的桶发现没有可用span，先把central cache的桶锁解掉，
    //这样如果其他线程释放内存对象回来，不会阻塞
    list._mtx.unlock();

    // 这里去page cache中获取一个span，pagecache的操作会同时影响多个桶和共享映射表，所以需要加全局锁
    PageCache::GetInstance()->_pageMtx.lock();
    //NewSpan:获取一个k页的span。NumMovePage(size)计算size大小的对象，PageCache计算一次向系统获取几个页
    Span* span = PageCache::GetInstance()->NewSpan(SizeClass::NumMovePage(size));
    span->_isUse = true;//标记span正在使用中，防止其他线程去pagecache合并内存时把这个span合并掉。
    PageCache::GetInstance()->_pageMtx.unlock();
```

```
33
34      // 对获取span进行切分，不需要加锁，因为这会其他线程访问不到这个span
35
36      // 计算span的大块内存的起始地址和大块内存的大小(字节数)
37      // span->_pageId是span的大块内存的起始页号，左移13位就是span的大块内存的起始地址
38      char* start = (char*)(span->_pageId << PAGE_SHIFT);
39      // span->_n是span的页数，bytes是span的大块内存的总字节数
40      size_t bytes = span->_n << PAGE_SHIFT;
41      //系统内存按页划分，第0页起始地址为0，第1页起始地址为8kb，第2页16kb。。。
42      //注意：这里的“第 0 页”只是整个虚拟地址空间的第 0 个页区间，并不表示你的程序真的能使
    用地址 0
43      // 页号：    98          99        100       101       102       103
44      //
    ┌──────────┬──────────┬──────────┬──────────┬──────────┬──────────┐
45      //        │  8KB     │  8KB     │  8KB     │  8KB     │  8KB     │  8KB     │
46      //
    └──────────┴──────────┴──────────┴──────────┴──────────┴──────────┘
47      //                                  └──────────── Span ────────────┘
48      //                                   _pageId=100, _n=3
49
50      char* end = start + bytes;
51      //假设现在申请到一个 span，它有 1 页，每页大小是 8KB = 8192 字节；当前要给
    CentralCache 分配的对象大小是 32 字节
52      // 那么这个 span 可以切分成 8192 / 32 = 256 个小对象
53      //假设这段内存的起始地址是 0x1000，即start = 0x1000，end = start + bytes =
    0x1000 + 8192 = 0x3000
54      //也就是说，可切分的内存范围是：[start, end)，即[0x1000, 0x3000)
55      //每块 32 字节：
56      // 第1块：0x1000 ~ 0x101F
57      // 第2块：0x1020 ~ 0x103F
58      // 第3块：0x1040 ~ 0x105F
59      // ...
60      // 第256块：0x2FE0 ~ 0x2FFF
61
62      // 把大块内存切成自由链表链接起来，先切一块下来去做头，方便尾插
63      //这里start和end是大块内存的起始地址和结束地址，size是小对象的大小
64      //tail是span->_freeList自由链表的最后一个对象，初始时指向第一个小对象
65
66      span->_freeList = start;// span->_freeList指针指向切好的小块内存的自由链表的第一
    个对象
67      start += size;// start指针向后移动size字节，指向下一个小对象的起始地址
68      void* tail = span->_freeList;// tail指针指向自由链表的最后一个对象，初始时指向第
    一个小对象
69      int i = 1;
70      //之后从 start 开始，每次移动 size 个字节，把整段内存切成大小为 size 的小对象，
71      //并用每个对象的开头空间存放下一个对象的地址：
72      while (start < end)
```

```
73      {
74          ++i;
75          NextObj(tail) = start;
76          tail = NextObj(tail);
77          start += size;
78      }
79      NextObj(tail)=nullptr;
80
81      // 切好span以后，需要把span挂到桶里面去的时候，再加锁
82      //切分过程中，这个新 span 尚未放入 CentralCache 桶，其他线程无法通过该桶找到它，所以切分本身不需要持有桶锁。切分完成后再加锁，把 span 插入链表头部，然后返回。
83      list._mtx.lock();
84      list.PushFront(span);
85
86      return span;
87  }
```

4.3CentralCache释放内存

当 ThreadCache 的自由链表过长时，会通过：ReleaseListToSpans();把一段对象链表归还给 CentralCache。

此时有一个问题：ThreadCache 只知道对象地址，并不知道这个对象原来属于哪个 Span。

所以项目在 PageCache 中维护了一个页号到 Span 的哈希映射：

代码块

```
1   std::unordered_map<PAGE_ID, Span*> _idSpanMap;
```

CentralCache 根据对象地址计算页号：

代码块

```
1   //项目中 PAGE_SHIFT 等于13，因为一页是：2^13B = 8KB
2   PAGE_ID id = (PAGE_ID)obj >> PAGE_SHIFT;
```

得到页号以后，通过 `_idSpanMap` 找到对象所属的 Span。

然后把对象重新头插到这个 Span 的 `_freeList` 中，同时将 `_useCount` 减一。

如果 `_useCount` 仍然大于0，说明这个 Span 还有一部分对象在 ThreadCache 或者用户手中，因此 Span 继续留在 CentralCache。

如果 `_useCount` 变为0，说明这个 Span 切分出去的所有小对象都已经归还。

这时 CentralCache 会把 Span 从当前 SpanList 中删除，然后释放 CentralCache 的桶锁，再获取 PageCache 的全局锁，把整个 Span 归还给 PageCache。

代码块

```
//threadcache还内存对象（一段以start为开头，大小size的自由链表）给centralcache
//除了要确定这个自由链表要挂到哪个桶上，还需要确定这个自由链表的对象是属于centralcache中的哪个span
//知道要还回来的内存对象地址，可以先算出属于哪个页，然后根据页和span的映射关系找到对应span
//把要还的对象重新头插到span的freelist上
void CentralCache::ReleaseListToSpans(void* start, size_t size)
{
    size_t index = SizeClass::Index(size);//根据要还的对象大小算出要还的桶号位置
    //上桶锁
    _spanLists[index]._mtx.lock();
    while (start)
    {
        void* next = NextObj(start);

        //根据内存对象地址，算出对应的页号，然后查找页号对应的span
        Span* span = PageCache::GetInstance()->MapObjectToSpan(start);
        //把内存对象挂到span的自由链表上（头插）
        NextObj(start) = span->_freeList;
        span->_freeList = start;
        span->_useCount--;

        // 说明span的切分出去的所有小块内存都回来了
        // 这个span就可以再回收给page cache，pagecache可以再尝试去做前后页的合并
        if (span->_useCount == 0)
        {
            _spanLists[index].Erase(span);
            span->_freeList = nullptr;
            span->_next = nullptr;
            span->_prev = nullptr;

            // 释放span给page cache时，使用page cache的锁就可以了
            // 这时把桶锁解掉
            _spanLists[index]._mtx.unlock();

            PageCache::GetInstance()->_pageMtx.lock();
            PageCache::GetInstance()->ReleaseSpanToPageCache(span);
            PageCache::GetInstance()->_pageMtx.unlock();

            _spanLists[index]._mtx.lock();
        }

```

```
            start = next;
        }

        _spanLists[index]._mtx.unlock();
    }


    //根据内存对象地址，算出对应的页号，然后查找页号对应的span
    Span* PageCache::MapObjectToSpan(void* obj)
    {
        //地址转换成页号
        PAGE_ID id = ((PAGE_ID)obj >> PAGE_SHIFT);
        //根据页号找到对应span
        std::unique_lock<std::mutex> lock(_pageMtx);
        auto ret = _idSpanMap.find(id);

        if (ret != _idSpanMap.end())
        {
            return ret->second;
        }
        else
        {
            assert(false);
            return nullptr;
        }
    }
```

4.4CentralCache核心逻辑

所以 CentralCache 的核心逻辑可以总结为：

代码块

```
申请时：
从 Span 中批量分发小对象
没有可用 Span 就向 PageCache 申请页

释放时：
小对象回到原 Span
Span 完全空闲后归还 PageCache
```

五、PageCache

PageCache 是三级缓存的最上层，它不再按照对象大小管理内存，而是按照页数管理 Span。

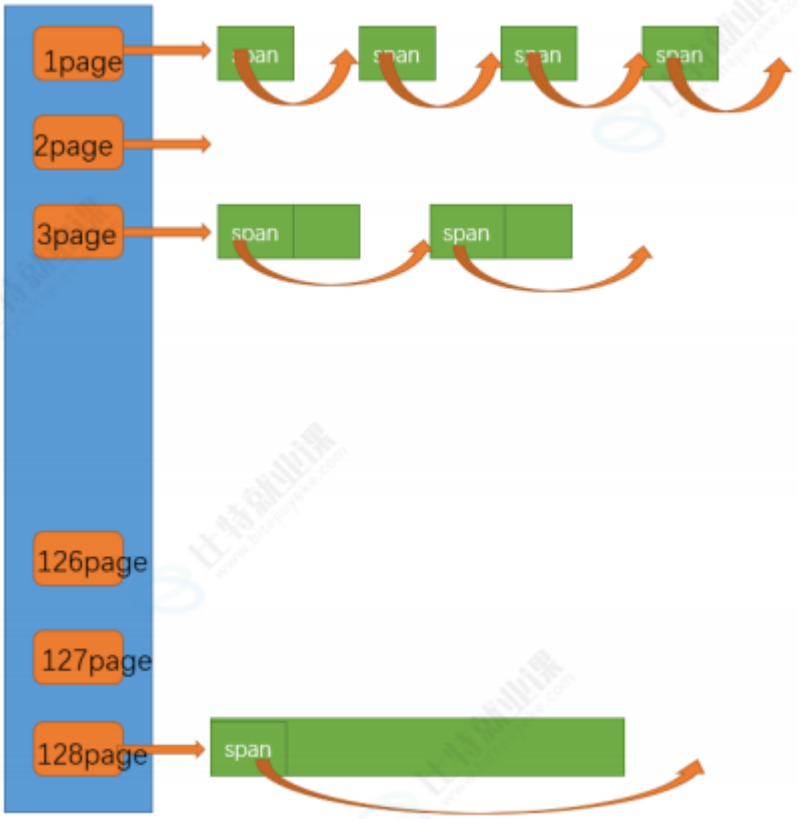
项目中一页为8KB，PageCache 包含129个 SpanList。其中0号桶不使用：

代码块

```
第1号桶存放1页的空闲 Span
第2号桶存放2页的空闲 Span
.....
第128号桶存放128页的空闲 Span
```

PageCache 主要负责四件事：

- 为 CentralCache 提供连续页；
- 将大 Span 切分成小 Span；
- 回收时合并相邻空闲 Span；
- 处理大于256KB的内存申请。



5.1PageCache全局锁设计

CentralCache 可以使用桶锁，但是 PageCache 使用的是一把全局锁。

原因是 PageCache 的一次操作可能同时影响多个桶。

例如申请5页时，第5号桶为空，但是第10号桶有一个 Span。PageCache 需要：

1. 从第10号桶删除这个 Span；
2. 从中切出5页；

3. 把剩余5页放入第5号桶；
4. 更新多个页号到 Span 的映射。

释放时也可能同时删除前后相邻 Span、修改多个桶并更新映射表。

因此，PageCache 的操作属于跨桶操作，使用全局锁更容易保证 SpanList 和 `_idSpanMap` 的一致性。

5.2PageCache申请内存

PageCache 申请 Span 的主要接口是：PageCache::NewSpan(k);

其中 `k` 表示需要的页数。

首先会判断要申请的内存是否大于128页（1页8kb），如果大于则直接向堆申请，

如果小于则是如下几种情况：

第一种情况，如果第k号桶中有空闲 Span，就直接从该桶头部取出一个 Span 返回。

第二种情况，如果第k号桶为空，就向后查找更大的 Span。

例如申请5页，第5号桶为空，但是第10号桶有一个 Span，那么就从这个10页 Span 的头部切出5页返回，把剩余的5页重新放到第5号桶。

第三种情况，如果从第k号桶一直到第128号桶都没有可用 Span，说明 PageCache 中没有足够的空闲页。会调用

SystemAlloc(128);直接向堆申请128页内存，然后挂到128号桶上以供后续切割。

代码块

```
// 获取一个K页的span
Span* PageCache::NewSpan(size_t k)
{
    //要申请的内存>128页，向堆要
    if(k>NPAGES-1)
    {
        void* ptr=SystemAlloc(k);
        //Span* span=new Span;
        Span* span = _spanPool.New();
        span->_pageId=(PAGE_ID)ptr>>PAGE_SHIFT;
        span->_n=k;
        _idSpanMap[span->_pageId]=span;

        return span;
    }

    assert(k > 0 && k < NPAGES);
```

```
18
19      // 先检查第k个桶里面有没有span
20      if (!_spanLists[k].Empty())
21      {
22          return _spanLists[k].PopFront();
23      }
24
25      // 检查一下后面的桶里面有没有span，如果有可以把它进行切分
26      for (size_t i = k+1; i < NPAGES; ++i)
27      {
28          //pagecache某个桶里的大块span不为空，进行切分
29          if (!_spanLists[i].Empty())
30          {
31              //头出一个大块span
32              Span* nSpan = _spanLists[i].PopFront();
33              //Span* kSpan = new Span;
34              Span* kSpan = _spanPool.New();
35
36              // 在nSpan的头部切一个k页下来
37              // k页span返回给centracache
38              // n-k页的Span再挂到对应映射的位置
39              kSpan->_pageId = nSpan->_pageId;
40              kSpan->_n = k;
41
42              nSpan->_pageId += k;
43              nSpan->_n -= k;
44
45              //切剩下的挂到pagecache对应页的桶上
46              _spanLists[nSpan->_n].PushFront(nSpan);
47              // 存储nSpan的首尾页号跟nSpan映射，方便page cache回收内存时进行的合并查找
48              _idSpanMap[nSpan->_pageId] = nSpan;
49              _idSpanMap[nSpan->_pageId + nSpan->_n - 1] = nSpan;
50
51              // 建立id和span的映射，方便central cache回收小块内存时，查找对应的span
52              for (PAGE_ID i = 0; i < kSpan->_n; ++i)
53              {   //建立要分配出去的页号和kspan的映射，方便central cache回收小块内存
    时，查找对应的span
54                  //也就是分配出去的对应的页属于哪个span
55                  _idSpanMap[kSpan->_pageId + i] = kSpan;
56              }
57
58              return kSpan;
59          }
60      }
61
62      // 走到这个位置就说明后面没有大页的span了，一般是刚开始的时候，pagecache还没有从堆上
    申请过span，或者是page cache里面的span都被central cache分配完了
```

```
63      //Span* bigSpan = new Span;
64      Span* bigSpan = _spanPool.New();
65      // 这时就去找堆要一个128页的span，插入到128号桶
66      void* ptr = SystemAlloc(NPAGES - 1);
67      bigSpan->_pageId = (PAGE_ID)ptr >> PAGE_SHIFT;
68      bigSpan->_n = NPAGES - 1;
69      _spanLists[bigSpan->_n].PushFront(bigSpan);
70
71      //再递归调一次，前面的逻辑就会把128页的span切分成k页的span返回给central cache
72      return NewSpan(k);
73  }
```

5.3PageCache释放内存

PageCache 回收 Span 时，首先判断 Span 是否超过128页。

如果超过128页，说明它原来就是直接向系统申请的，因此直接调用 `SystemFree` 归还操作系统，然后删除对应的 Span 元数据。

如果不超过128页，则不会直接放回对应桶，而是先尝试与前后相邻的空闲 Span 合并。

代码块

```
1   void PageCache::ReleaseSpanToPageCache(Span* span)
2   {
3       //释放大于128页的span，说明不是找内存池要的，直接释放给堆
4       if(span->_n > NPAGES - 1)
5       {
6           void* ptr = (void*)(span->_pageId << PAGE_SHIFT);// span->_pageId是span
    的大块内存的起始页号，左移13位就是span的大块内存的起始地址
7           SystemFree(ptr);
8           //delete span;
9           _spanPool.Delete(span);
10          return;
11      }
12
13      // 对span前后的页，尝试进行合并，缓解内存碎片(外碎片)问题
14      //因为如果不合并，返回的很多都是较小页的span，
15      //如果想要大块内存，就又需要去堆上重新申请，增加了堆的压力
16      while (1)
17      {
18          PAGE_ID prevId = span->_pageId - 1;//获取当前span前1页的pageid
19          auto ret = _idSpanMap.find(prevId);//根据页号找到对应span
20          // 前面的页号没有，不合并了
21          if (ret == _idSpanMap.end())
22          {
23              break;
```

```
24          }
25
26          // 前面相邻页的span在使用，不合并了
27          Span* prevSpan = ret->second;
28          if (prevSpan->_isUse == true)
29          {
30              break;
31          }
32
33          // 合并出超过128页的span没办法管理，不合并了
34          //这里是有可能的，比如前一个申请了128页的span，后一个申请了1页的span，这两个
    span的页号正好相邻
35          if (prevSpan->_n + span->_n > NPAGES-1)
36          {
37              break;
38          }
39
40          span->_pageId = prevSpan->_pageId;//合并完，前一个span的页号就是大块内存页号
41          span->_n += prevSpan->_n;//合并完，前一个span的页数加上当前span的页数就是大
    块内存页数
42
43          _spanLists[prevSpan->_n].Erase(prevSpan);
44          //delete prevSpan;
45          _spanPool.Delete(prevSpan);
46      }
47
48      // 向后合并
49      while (1)
50      {
51          PAGE_ID nextId = span->_pageId + span->_n;//获取当前span后n页的pageid，当
    前span共n页
52          auto ret = _idSpanMap.find(nextId);//根据页号找到对应span
53          // 后面的页号没有，不合并了
54          if (ret == _idSpanMap.end())
55          {
56              break;
57          }
58          // 后面相邻页的span在使用，不合并了
59          Span* nextSpan = ret->second;
60          if (nextSpan->_isUse == true)
61          {
62              break;
63          }
64          // 合并出超过128页的span没办法管理，不合并了
65          if (nextSpan->_n + span->_n > NPAGES-1)
66          {
67              break;
```

```
68            }
69            // 合并完页数加上后一个span的页数就是大块内存页数，原先span起始页号不需要改
70            span->_n += nextSpan->_n;
71
72            _spanLists[nextSpan->_n].Erase(nextSpan);
73            //delete nextSpan;
74            _spanPool.Delete(nextSpan);
75        }
76
77        // 合并完后，把span挂到pagecache对应页的桶上
78        _spanLists[span->_n].PushFront(span);
79        //合并完的span标记为未使用
80        span->_isUse = false;
81        // 建立id和span的映射，方便central cache回收小块内存时，查找对应的span
82        _idSpanMap[span->_pageId] = span;
83        _idSpanMap[span->_pageId+span->_n-1] = span;
84    }
```

例如，当前有一个5页 Span，前面有一个3页的空闲 Span，后面有一个2页的空闲 Span，那么最终可以合并成一个10页 Span，再放入第10号桶。

PageCache 合并相邻 Span，主要是为了缓解外碎片。

如果不进行合并，PageCache 中可能存在很多离散的小 Span。虽然总空闲页数很多，但在需要较大连续内存时仍然无法直接满足，只能再次向系统申请。

合并以后，小的连续空闲块可以重新组成更大的连续块，提高内存复用率。

六、项目的并发设计、解决的问题

这个项目的并发设计主要分成三个层次。

ThreadCache 是线程私有的，因此本地自由链表操作不需要加锁，这也是最高频的执行路径。

CentralCache 是线程共享的，但是使用的是桶锁。不同大小类别的对象可以并行申请和释放，只有相同大小类别之间才会产生锁竞争。

PageCache 使用全局锁，因为它的切分、合并和页号映射更新都是跨桶操作，需要保证整个状态的一致性。

所以这个项目不是完全无锁，而是：

> 尽量让最高频的小对象操作走 ThreadCache 无锁路径，同时通过 CentralCache 桶锁和 PageCache 全局锁保证共享数据安全。

这个项目主要解决了三个问题。

第一个是锁竞争问题。

通过TLS为每个线程提供独立的ThreadCache，使高频小对象申请和释放尽量在线程本地完成。CentralCache 使用桶锁，进一步缩小锁的粒度。

第二个是系统调用开销问题。

ThreadCache 和 CentralCache 之间采用批量移动，CentralCache 和 PageCache 之间以 Span 为单位移动，PageCache 向系统一次申请128页，避免每次小对象申请都直接调用系统接口。

第三个是内存碎片问题。

对于内碎片，项目通过分段对齐规则，把浪费控制在大约10%左右。

对于外碎片，PageCache 在释放 Span 时会合并前后相邻的空闲 Span，形成更大的连续内存块。

七、BenchMark测试提升性能对比

代码块

```
void BenchmarkMalloc(size_t ntimes, size_t nworks, size_t rounds)
{
    std::vector<std::thread> vthread(nworks);
    std::atomic<size_t> malloc_costtime = 0;
    std::atomic<size_t> free_costtime = 0;

    for (size_t k = 0; k < nworks; ++k)
    {
        vthread[k] = std::thread([&, k]() {
            std::vector<void*> v;
            v.reserve(ntimes);

            for (size_t j = 0; j < rounds; ++j)
            {
                size_t begin1 = clock();
                for (size_t i = 0; i < ntimes; i++)
                {
                    v.push_back(malloc(16));
                    //v.push_back(malloc((16 + i) % 8192 + 1));
                }
                size_t end1 = clock();

                size_t begin2 = clock();
                for (size_t i = 0; i < ntimes; i++)
                {
                    free(v[i]);
                }
                size_t end2 = clock();
                v.clear();
```

```
30
31                  malloc_costtime += (end1 - begin1);
32                  free_costtime += (end2 - begin2);
33              }
34          });
35      }
36
37      for (auto& t : vthread)
38      {
39          t.join();
40      }
41
42      printf("%zu个线程并发执行%zu轮次，每轮次malloc %zu次: 花费: %zu ms\n",
43          nworks, rounds, ntimes, malloc_costtime.load());
44
45      printf("%zu个线程并发执行%zu轮次，每轮次free %zu次: 花费: %zu ms\n",
46          nworks, rounds, ntimes, free_costtime.load());
47
48      printf("%zu个线程并发malloc&free %zu次，总计花费: %zu ms\n",
49          nworks, nworks * rounds * ntimes,
50          malloc_costtime.load() + free_costtime.load());
51  }
52
53  // 单轮次申请释放次数 线程数 轮次
54  void BenchmarkConcurrentMalloc(size_t ntimes, size_t nworks, size_t rounds)
55  {
56      std::vector<std::thread> vthread(nworks);
57      std::atomic<size_t> malloc_costtime = 0;
58      std::atomic<size_t> free_costtime = 0;
59
60      for (size_t k = 0; k < nworks; ++k)
61      {
62          vthread[k] = std::thread([&]() {
63              std::vector<void*> v;
64              v.reserve(ntimes);
65
66              for (size_t j = 0; j < rounds; ++j)
67              {
68                  size_t begin1 = clock();
69                  for (size_t i = 0; i < ntimes; i++)
70                  {
71
72                      v.push_back(ConcurrentAlloc(16));
73                      //v.push_back(ConcurrentAlloc((16 + i) % 8192 + 1));
74                  }
75                  size_t end1 = clock();
76
```

```
                size_t begin2 = clock();
                for (size_t i = 0; i < ntimes; i++)
                {
                    ConcurrentFree(v[i],16);
                }
                size_t end2 = clock();
                v.clear();

                malloc_costtime += (end1 - begin1);
                free_costtime += (end2 - begin2);
            }
        });
    }

    for (auto& t : vthread)
    {
        t.join();
    }

    printf("%zu个线程并发执行%zu轮次，每轮次concurrent alloc %zu次: 花费: %zu ms\n",
        nworks, rounds, ntimes, malloc_costtime.load());

    printf("%zu个线程并发执行%zu轮次，每轮次concurrent dealloc %zu次: 花费: %zu ms\n",
        nworks, rounds, ntimes, free_costtime.load());

    printf("%zu个线程并发concurrent alloc&dealloc %zu次，总计花费: %zu ms\n",
        nworks, nworks * rounds * ntimes,
        malloc_costtime.load() + free_costtime.load());
}

int main()
{
    size_t n = 1000;
    cout << "==========================================================" << endl;
    BenchmarkConcurrentMalloc(n, 4, 10);
    cout << endl << endl;

    BenchmarkMalloc(n, 4, 10);
    cout << "==========================================================" << endl;

    return 0;
}
```

测试结果如下：

测试1：每次申请固定16字节

==========

4个线程并发执行10轮次，每轮次concurrent alloc 1000次: 花费：8 ms

4个线程并发执行10轮次，每轮次concurrent dealloc 1000次: 花费：10 ms

4个线程并发concurrent alloc&dealloc 40000次，总计花费：18 ms

4个线程并发执行10轮次，每轮次malloc 1000次: 花费：86 ms

4个线程并发执行10轮次，每轮次free 1000次: 花费：32 ms

4个线程并发malloc&free 40000次，总计花费：118 ms

==========

测试2：4个线程并发执行，每个线程重复10轮，每轮按照固定顺序申请和释放17B～1016B的1000个小对象

==========

4个线程并发执行10轮次，每轮次concurrent alloc 1000次: 花费：43 ms

4个线程并发执行10轮次，每轮次concurrent dealloc 1000次: 花费：38 ms

4个线程并发concurrent alloc&dealloc 40000次，总计花费：81 ms

4个线程并发执行10轮次，每轮次malloc 1000次: 花费：2202 ms

4个线程并发执行10轮次，每轮次free 1000次: 花费：1860 ms

4个线程并发malloc&free 40000次，总计花费：4062 ms

==========